Grupo Viernes 8:00-10:00 semanas B

– Práctica 2 –

Autor: Romeo Puch NIP:925301

Autor: Javier Sánchez Mir NIP:926247

Ejercicio 1

1. Explicación

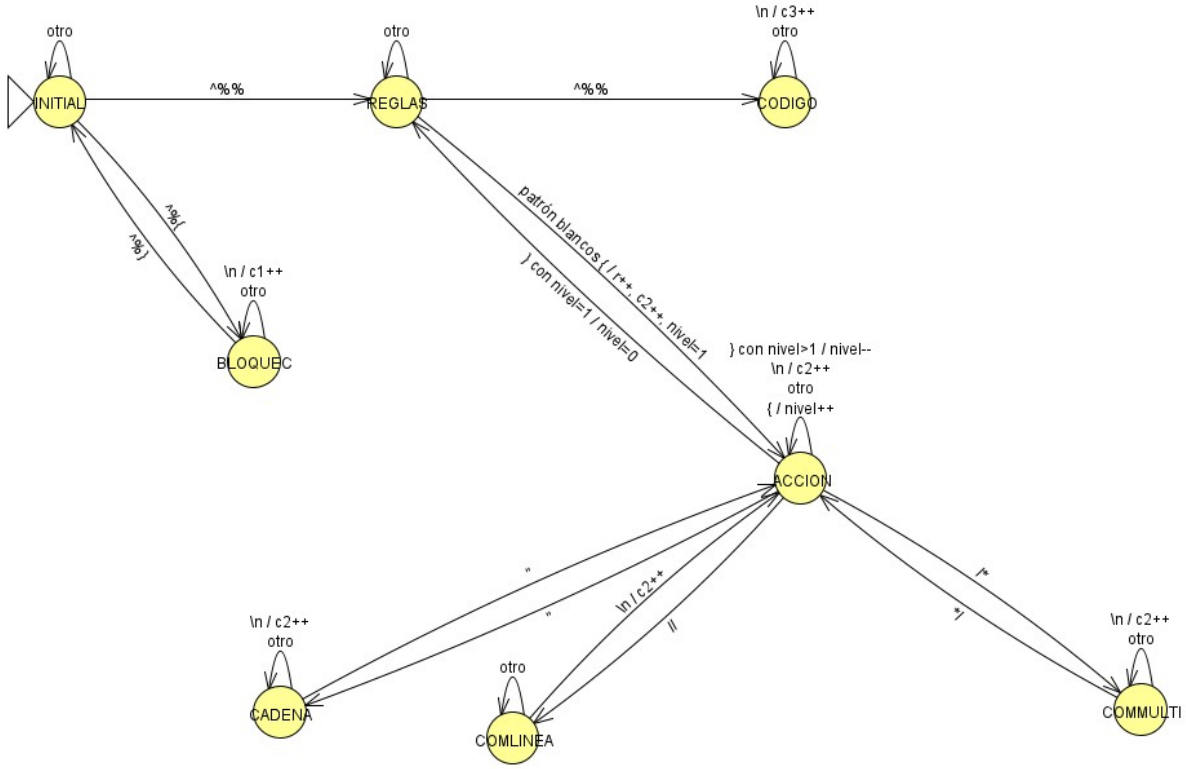
Para resolver este ejercicio hemos usado las condiciones de arranque de Flex. Hemos definido un estado exclusivo (`%x`) para cada contexto en el que puede encontrarse el analizador mientras recorre el fichero de entrada, de forma que en cada estado solo se aplican las reglas que le corresponden.

Los estados que hemos usado son los siguientes:

- **INITIAL**: sección de declaraciones. Se ignora todo salvo `%{` al inicio de línea, que nos lleva a BLOQUEC, y `%%` al inicio de línea, que nos lleva a REGLAS.
- **BLOQUEC**: dentro de un bloque `%{ … %}`. Cada salto de línea suma uno a C1. Solo cierra el bloque un `%}` al inicio de línea; así, los `%}` que aparecen dentro de un comentario, como en el ejemplo del enunciado, no lo cierran, para ello se ha usado la condicion `^`.
- **REGLAS**: sección de reglas. Como los patrones no contienen blancos, detectamos el inicio de una regla con el patrón `^[^[:space:]]+[[:blank:]]+"{"`, es decir, el patrón de la regla, uno o más blancos y la llave que abre la acción. Cada vez que se reconoce sumamos uno a R y uno a C2 (el “+1” de cada acción), ponemos el nivel de llaves a 1 y pasamos a ACCION. De esta forma, las llaves que forman parte del patrón (por ejemplo `{DIGITO}+`) no se confunden con las de la acción.
- **ACCION**: dentro de una acción. Cada salto de línea suma uno a C2. Usamos una variable `nivel` para seguir las llaves anidadas (bloques if, for…): `{` la incrementa y `}` la decrementa, y cuando llega a 0 la acción ha terminado y volvemos a REGLAS.
- **CADENA, COMLINEA y COMMULTI**: dentro de una cadena, de un comentario `//` o de un comentario `/* */`. En estos estados las llaves no se tienen en cuenta, pero los saltos de línea sí se suman a C2 porque siguen formando parte de la acción. Salimos de ellos con `"`, con el salto de línea y con `*/`, respectivamente.
- **CODIGO**: sección de código. Cada salto de línea suma uno a C3, empezando por el de la propia línea `%%`, que según el enunciado forma parte de esta sección.

Al terminar la lectura, el `main` imprime los cuatro contadores con el formato pedido. El programa puede leer el fichero tanto como argumento como por la entrada estándar.

En el siguiente diagrama se muestran los estados y las transiciones entre ellos. Cada transición indica lo que se lee y, tras la barra, lo que se hace. No es un autómata de aceptación, por lo que no tiene estados finales.



Ejercicio 1 — 2. Pruebas

Para comprobar que funciona correctamente hemos creado cuatro ficheros de prueba:

- `prueba_ej1_1.txt`: el ejemplo del enunciado.
- `prueba_ej1_2.txt`: un caso con comentarios multilínea dentro de los bloques C, llaves dentro de cadenas y de los dos tipos de comentario, y bloques if/for anidados dentro de una acción.
- `prueba_ej1_3.txt`: un fichero mínimo sin sección de código.
- `prueba_ej1_4.txt`: el propio fichero `ej1.l`, como caso grande y real.

Hemos ejecutado nuestro programa y el ejecutable correcto proporcionado con cada prueba, y hemos comparado ambas salidas con diff. En ningún caso ha impreso nada, lo que significa que las salidas son idénticas.

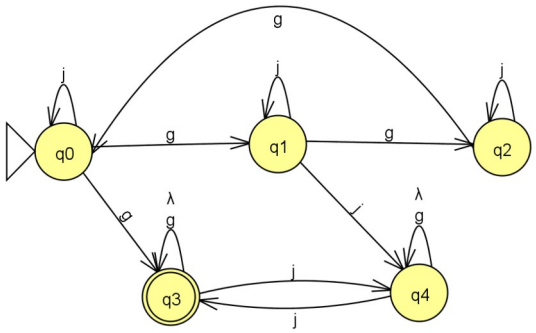
Ejercicio 2

1. Explicación

Para resolver este problema, siguiendo los consejos del guión de la práctica y los conocimientos de la teoría de clase, hemos diseñado un AFD que reconoce el lenguaje L = {w ∈ Σ ∗ | w = u1xu2, |u1x|g = 3n+ 1, |xu2|j = 2m u1, u2 ∈ Σ ∗ , x ∈ Σ, n, m ∈ N} dado. Para ello, hemos empezado diseñando un AFnD en JFLAP, después lo hemos convertido en un AFD y lo hemos minimizado.

El AFnD diseñado es el siguiente:

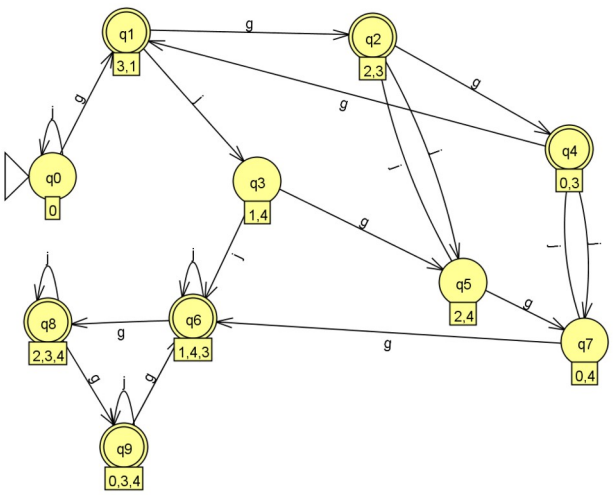
Estado	g	j
q0 (inicial)	q1 o q3	q0
q1	q2	q1 o q4
q2	q0	q2
q3 (final)	q3	q4
q4	q4	q3



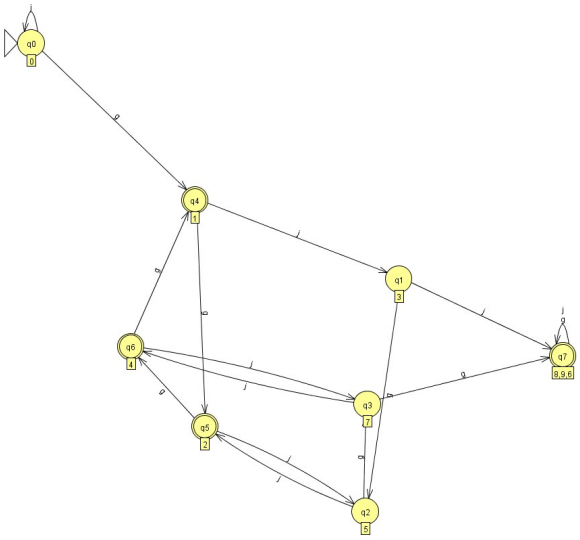
El AFnD determina cuál es la semilla marcada x de la definición de L.

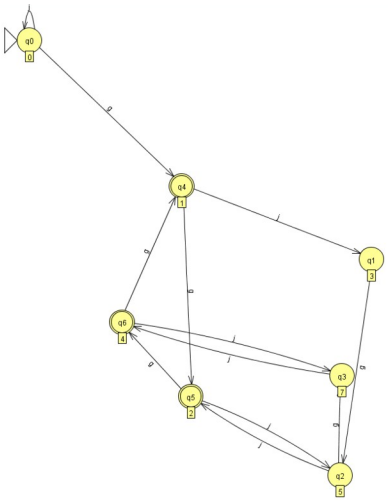
- **Antes de x (q0, q1, q2):** Cuentan las g 's módulo 3: q0 es resto 0, q1 resto 1 y q2 resto 2. Cada g avanza q0 → q1 → q2 → q0, y las j no cambian la cuenta (bucle). Así se controla la condición $|u_1x|_g = 3n+1$.
- **Elección de x:** Desde q0 con g, la cuenta pasa a 1, así que esa g puede ser x: pasa a q3, porque x es g y de momento no hay j 's (cantidad par). Desde q1 con j, la cuenta sigue siendo 1, así que esa j puede ser x: pasa a q4, porque ya hay una j (cantidad impar). Desde q2 nunca se puede elegir x.
- **Desde x hasta el final (q3, q4):** Cuentan la paridad de j 's: q3 es par y q4 impar. Las g's no cambian el estado y cada j lo alterna. Se acepta si termina en q3 (final), que es la condición $|xu_2|_j = 2m$.

Una vez hecha la conversión a AFD, nos queda:



Minimizamos:



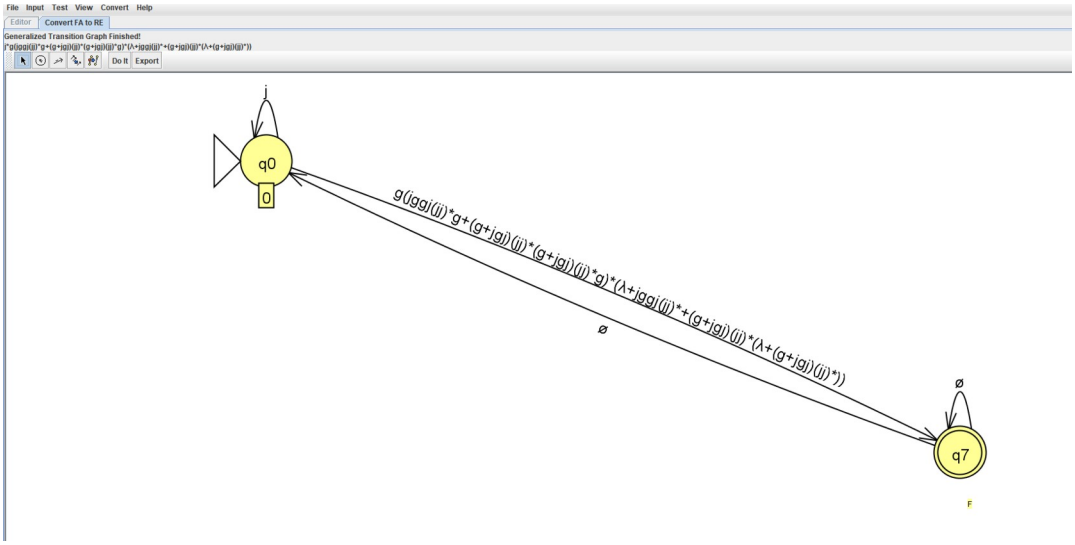


Sin ningún bonus, el jugador 1 gana siempre: empieza y ambos colocan el mismo número de semillas. Para ganar, el jugador 2 necesita ganar un tempo, y eso solo ocurre con el bonus de repetir turno, que depende de si la hilera pertenece a L (existe una semilla x que cumple las dos condiciones). Por tanto, hay que ver qué hileras h_0 dan ventaja al jugador 2.

El estado q7 es una trampa: es final, pero con cualquier símbolo se queda en q7. Desde ahí, cada semilla que se coloca da derecho a bonus, así que quien mueve (el jugador 1) puede colocar todas las suyas seguidas y ganar. Por eso, aunque q7 acepte, no favorece al jugador 2.

Por tanto, la expresión regular que se busca para el jugador 2 es la del autómata en el que los finales son solo q4, q5 y q6. Para obtenerla se eliminó q7 y se aplicó la conversión de JFLAP. La del jugador 1 es el complemento de esa expresión, que no hace falta escribir. En Flex basta con poner la regla del jugador 2 primero y, a continuación, una regla que capture cualquier otra línea (Flex elige el emparejamiento más largo y, en empate, la regla que aparece antes).

Al final, nos queda la expresión: `j*g(jggj(jj)*g+(g+jgj)(jj)*(g+jgj)(jj)*g)*(λ+jggj(jj)*+(g+jgj)(jj)*(λ+(g+jgj)(jj)*))`



Posteriormente, cambiamos los “+” por “|” para que Flex lo pueda leer. Como la cadena vacía “λ” tampoco la puede leer, usamos “?” de la siguiente manera: r? significa “r o nada” así que (λ + r) = (r)?, (λ + r + s) = (r | s)? Por lo que si sustituimos en nuestra expresión, nos queda:

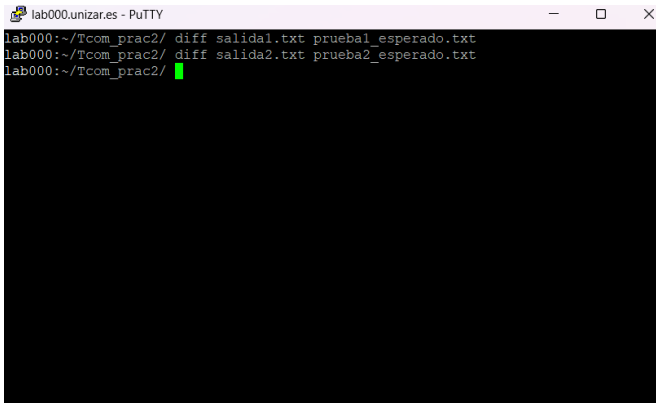
`j*g(jggj(jj)*g|(g|jgj)(jj)*(g|jgj)(jj)*g)*(jggj(jj)*|(g|jgj)(jj)*((g|jgj)(jj)*)?)?`, expresión que usamos en nuestro código.

Ejercicio 2 — 2. Pruebas

Para comprobar que funciona correctamente hemos creado dos ficheros de texto con las pruebas siguientes:

- **Primer fichero**: el mismo ejemplo del enunciado.
- **Segundo fichero**: 25 casos extra con cadenas cortas, largas, rachas de solo “g” o solo “j”, una línea vacía y algún caso que nos ha parecido interesante de probar.

Después, hemos creado otros dos ficheros con los resultados esperados de cada prueba y una vez compilado y ejecutado, hemos comparado con diff el fichero del resultado obtenido con el fichero del resultado esperado y diff no ha impreso nada, lo que significa que los ficheros son idénticos y por ende, el resultado es correcto.



Uso de herramientas de Inteligencia Artificial

Durante la realización de esta práctica hemos utilizado el asistente de inteligencia artificial Claude (Anthropic) con los siguientes fines:

- **Comprensión de la documentación:** como apoyo para entender los apartados del manual de Flex necesarios para la práctica, en especial el funcionamiento de las condiciones de arranque (`%x`, `BEGIN`) y de los patrones.
- **Dudas de sintaxis:** para resolver dudas concretas sobre la sintaxis de Flex y de C.
- **Compilación y pruebas:** como ayuda para compilar y ejecutar los programas y para comparar sus resultados con los ejecutables correctos al igual que para ayudar a corregir errores.
- **Memoria:** como apoyo en la redacción de las explicaciones, ayuda en la elaboración del diagrama de estados para hacerlo más visible en el documento y para organizar/mejorar la legibilidad del documento.